Autonomous Lidar Mapping of a Home with a Low-Cost Tracked Robot: Hector SLAM, Nav2 and Frontier Exploration on a Raspberry Pi 5

Marthens Hakizimana and Amir Taghavipour
Motion and Power Electronics Control (MPEC) Lab, Simon Fraser University
BCIT Centre for Welding Technologies and Metallurgy Research

October 17, 2026

Abstract

This paper describes how a low-cost educational tracked robot (Hiwonder MentorPi T1: Raspberry Pi 5, STM32 motor board, Oradar MS200 360° lidar, Deptrum Aurora 930 RGB-D camera) was turned into a robot that maps a house on its own. The robot runs the whole real-time loop on board: a lidar driver written for this work, Hector SLAM, the ROS 2 Nav2 stack (NavFn planner, Regulated Pure Pursuit controller, layered costmaps, recovery behaviours) and a frontier explorer that chooses where to go next. A laptop on the same Wi-Fi network only watches and steers: it shows the camera, the growing map, Nav2's plan, the frontiers and the goal, and lets the operator pause, take over with the keyboard, send the robot home and save the map. Obstacle avoidance is layered: the global planner avoids mapped walls, the controller's local costmap and collision check avoid what the lidar sees now, and the robot's own motor node refuses to drive into anything closer than 0.25 m whatever it is told. Bringing the system up on the real robot exposed problems that no simulation would have shown, the most important being that the motor signs had been calibrated wrong: the robot drove backwards and turned the wrong way while its wheel odometry reported the opposite, which corrupted the map within seconds. The error was found by matching lidar scans before and after a turn and confirmed with the camera. In an unattended run the robot mapped 47 m^2 of a home in 13 min, driving 74 m to 30 frontier goals (1 failed) and drove back to within 0.2 m of its start without any operator intervention; the frontier search took 50 ms on the Pi and the whole stack used about 1.5 of the Pi's four cores.

1 Introduction

Mapping an indoor space is the first step for almost every mobile-robot task: a map is what a robot localises against, plans in and reports on. With a 2D lidar the mapping itself is a solved problem [1, 2, 3]; what remains costly is the driving. An operator has to steer the robot slowly through every room, close loops and come back to fill gaps, and the quality of the map depends on how patiently that is done. Frontier-based exploration [8] automates this: the robot repeatedly drives to the boundary between what it has seen and what it has not, until no reachable boundary is left.

This work implements that loop on a low-cost tracked robot and documents the implementation in enough detail to be repeated. Its contributions are practical rather than algorithmic:

1. A complete, on-board ROS 2 Jazzy exploration stack for a Raspberry Pi 5 robot with a low-cost lidar: driver, SLAM, Nav2 configuration, a velocity adapter for a controller that takes fractions of maximum speed, and a frontier explorer with progress monitoring, blacklisting, return-to-home and run logging (sections 4 and 5).
2. A distributed split between robot and operator PC, chosen so that a Wi-Fi drop costs only the view, never the control loop, with the trade-offs measured (section 6).
3. A calibration procedure that uses only the lidar: the robot's true rotation is measured by circular cross-correlation of two scans, which revealed inverted motor signs that the wheel odometry could not see (section 7).
4. An offline simulation of the explorer and a real exploration of a home, with coverage, distance, timing and on-board load reported (section 8).

2 Related work

Lidar SLAM. Grid-based Rao–Blackwellised particle filters (GMapping [2]) need odometry and many particles; Hector SLAM [1] matches each scan against a multi-resolution occupancy map by Gauss–Newton optimisation and needs no odometry at all, which suits robots whose wheel odometry is poor, such as tracked robots that skid when they turn. Cartographer [3] and SLAM Toolbox [4] add pose-graph loop closure, at a higher computational cost. Hector was chosen here because the robot's track odometry is unreliable (section 7) and because its cost is a few percent of one Pi core.

Navigation. Nav2 [5] is the ROS 2 navigation framework: a behaviour tree sequences a global planner, a local controller and recovery behaviours, all working on layered costmaps [6]. Regulated Pure Pursuit [7] follows the global path with a lookahead point and slows down in tight spaces and near obstacles, which suits slow differential and tracked robots.

Exploration. Yamauchi's frontier exploration [8] drives to the nearest frontier. Later work weighs information gain against travel cost [9, 10], uses sampling (RRT-based exploration [11]) or learned policies. ROS packages such as

Table 1: robot04 and the operator PC.

Computer	Raspberry Pi 5, 8 GB, Ubuntu 24.04, ROS 2 Jazzy
Motor board	Hiwonder RRC Lite (STM32), own firmware, encoder speed loop
Drive	2 tracks, JGB37-520 1:90; top speed used 0.153 m/s (50 % of motor maximum)
Lidar	Oradar MS200: 360°, 10 Hz, 0.03–12 m, USB serial 230 400 baud
Camera	Deptrum Aurora 930: RGB 640×400 + aligned depth, 15 fps
Battery	2S Li-ion, 7.0–8.4 V
Operator PC	Laptop, Ubuntu 22.04, ROS 2 Humble, same Wi-Fi network
Middleware	Cyclone DDS on both machines, domain 17

`explore_lite` [12] implement greedy frontier exploration on top of Nav2. The explorer here is a greedy one with a capped size bonus, written so that every decision is logged and could be replayed.

3 Hardware

Table 1 lists the robot. The MentorPi T1 is a tracked chassis with two JGB37-520 geared motors (1:90, 3960 encoder counts per output turn) driven by an STM32 board (RRC Lite) that runs its own speed loop; the firmware used here was built from source and flashed from the Pi in an earlier lab [13]. The lidar sits 0.12 m above the floor on the robot's centre; the RGB-D camera looks forward 0.07 m ahead of it.

4 System architecture

Figure 1 shows the nodes and where they run. Everything that closes a control loop runs on the robot; the PC runs only the operator interface.

4.1 Drivers (boot service)

ms200_node. The MS200 streams 47-byte packets (header `0x54 0x2C`, rotation speed, start angle, 12 points of range and intensity, end angle, time stamp, CRC-8 with polynomial 0x4D) at 230 400 baud. No ROS 2 Jazzy driver was available, so one was written: a reader thread checks the CRC, interpolates the angle of each of the 12 points between the start and end angle, detects the end of a revolution when the angle wraps, and bins each revolution into 450 fixed rays (0.8°), keeping the nearest return per ray. Device angles are clockwise and are negated, so the published `LaserScan` is counter-clockwise from the robot's front as ROS expects. Returns closer than 0.10 m are dropped: the robot's own frame appears at 0.03–0.09 m. A udev rule matching both the USB IDs and the USB port gives the lidar a stable name (`/dev/lidar`), because the robot's voice module uses the same CH340 serial chip.

robot_node. Takes `cmd_vel` with `linear.x` and `angular.z` as fractions $-1 \dots 1$ of the configured maximum speed (not m/s), mixes them per track (left $= f - t$, right $= f + t$), and publishes encoder odometry and the TF `robot04/odom`→`robot04/base_footprint` at 20 Hz. It stops 0.5 s after the last command, and before and while driving it checks the lidar sector ±25° around the direction of travel: closer than 0.25 m it refuses to move. That check is independent of everything above it and is the last layer of obstacle avoidance.

Camera. The vendor driver publishes RGB and depth; `camera_bridge` turns them into a 480-pixel JPEG stream (10 Hz, only while someone subscribes). The driver occasionally stops delivering frames, so the bridge restarts it after 6 s without a frame.

4.2 SLAM

`hector_mapping` (the ROS 2 port by RobotX Workshops) builds a 1024×1024 grid at 5 cm (51 m square) with two resolution levels and publishes `map`→`robot04/odom`, so that the encoder odometry remains the child transform and Nav2 can use both. It is configured with a range window of 0.12–8 m and updates the map every 0.2 m or 0.06 rad.

4.3 Navigation

Nav2's controller, planner, behaviour, velocity-smoother and BT-navigator servers are started directly (not through `nav2_bringup`) with one parameter file. The main choices:

- **Footprint**: a circle of radius 0.15 m (body ≈ 0.24 × 0.20 m).
- **Global costmap** (map frame): Hector's map as static layer, the live scan as obstacle layer, inflation 0.45 m, unknown space tracked. **NavFn** plans with A* and `allow_unknown`, because frontier goals sit at the edge of the unknown.
- **Local costmap** (odom frame, rolling 3×3 m, 10 Hz): live scan only, inflation 0.40 m. Its update rate must not be lower than the controller rate (section 9).
- **Controller**: Regulated Pure Pursuit at 8 Hz, 0.10 m/s, lookahead 0.25–0.6 m, rotate-to-heading at 0.6 rad/s when the path turns more than 0.6 rad, linear speed regulated by curvature and by costmap cost, and a collision check that stops the robot if the path up to the carrot would collide within 1.5 s.
- **Goal checker**: 0.20 m, any heading (the lidar sees all round).
- **Transform tolerance** 1.0 s, because Hector stamps `map`→`odom` with the time of the scan it was computed from, 0.1–0.2 s in the past.

4.4 Velocity adapter

Nav2 commands v (m/s) and ω (rad/s); robot_node expects fractions. With the measured top speed $v_{\max} = 0.153$ m/s and effective track width $b = 0.105$ m (section 7),

$$f = \frac{v}{v_{\max}}, \qquad t = \frac{\omega b}{2 v_{\max}}, \qquad \ell = f - t, \quad r = f + t. \tag{1}$$

The motor speed loop does not move a track below about 0.1 of its maximum, so each non-zero track fraction is raised to at least 0.12 (with its sign), and both are scaled down together if one exceeds 1, which keeps the curvature. The adapter is muted until the explorer starts and whenever the

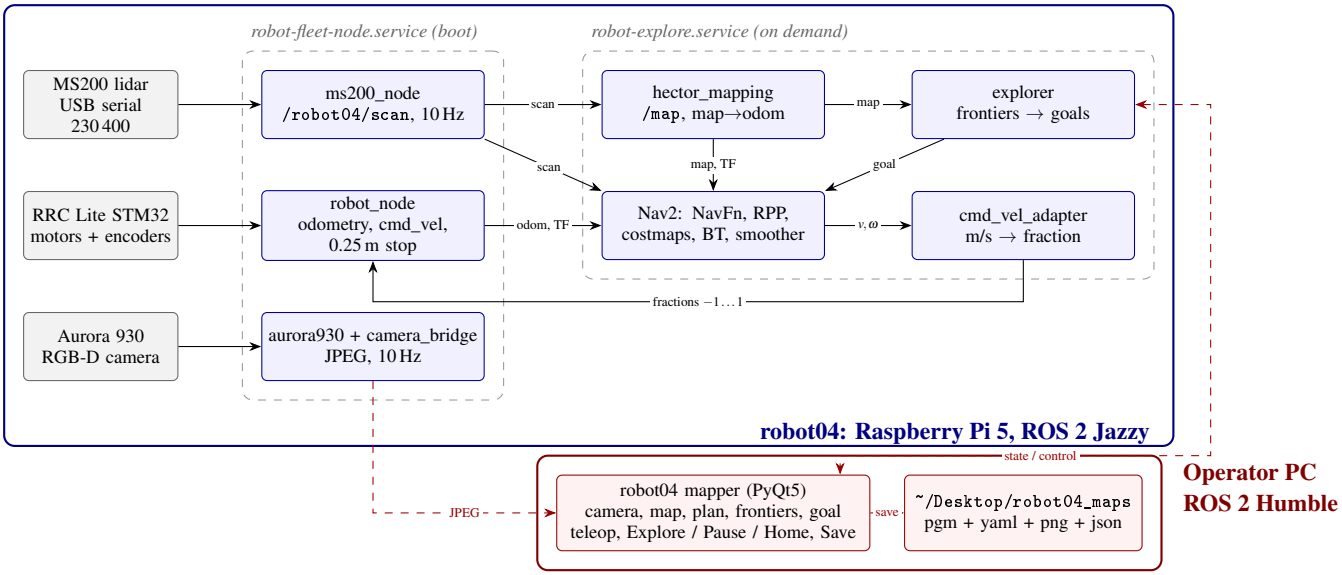


Figure 1: Nodes and their placement. Solid arrows stay on the robot; dashed arrows cross Wi-Fi. The robot's boot service runs the drivers; the exploration service is started from the PC when needed. The operator PC never sits inside a control loop.

operator pauses, so manual teleop and Nav2 never command the motors at the same time.

5 Frontier explorer

The explorer is a 400-line Python node. Once per second it reads the robot pose from TF (`map`→`base_footprint`) and every 3 s it searches the latest map.

Frontier search. Cells are free ($0 \le p \le 25$), occupied ($p \ge 65$) or unknown (-1). The search works on the known part of the map only (a margin of 20 cells around it), because the 1024×1024 grid is mostly unknown. Steps:

1. Clearance: Euclidean distance transform of the non-occupied cells, in metres.
2. Traversable cells: free cells with clearance $\ge$ the robot radius (0.15 m).
3. Reachability: 8-connected Dijkstra from the traversable cell nearest the robot gives the geodesic distance d to every reachable cell.
4. Frontier cells: free cells with an unknown 4-neighbour, grouped into 8-connected clusters; clusters shorter than 0.30 m are ignored.
5. Goal of a cluster: among reachable cells within 0.35 m of the cluster, the one nearest its centroid. Clusters with no such cell are unreachable and skipped.
6. Score $= d_{\text{goal}} - 0.5\,\min(L, 1.5\,\text{m})$, with L the cluster length. The lowest score wins: the nearest frontier, with a bonus for larger openings. The bonus is capped because the ragged edge of a long scan can merge into one cluster tens of metres long, whose bonus would otherwise dominate (observed in the first run: a 38.9 m "frontier").

Goal handling. The winner becomes a `NavigateToPose` goal facing away from the robot. While it is active, each new search checks whether a frontier still exists within 0.6 m of the goal's frontier centroid; if not, the area has been seen on the way and the goal is cancelled in favour of the next one ("explored"). A goal fails when Nav2 aborts it, when the robot has not moved 0.15 m or turned 0.5 rad for 30 s, or after a timeout of $40\,\text{s} + 2.5\,d/0.05\,\text{m/s}$. Failed goals and their frontiers are blacklisted (0.4 m radius). Goals that are *reached* are blacklisted too: a frontier that is still there when the robot stands next to it is a pocket the lidar cannot see into, typically under furniture, and without this rule the robot re-sent the same goal 39 times in two minutes.

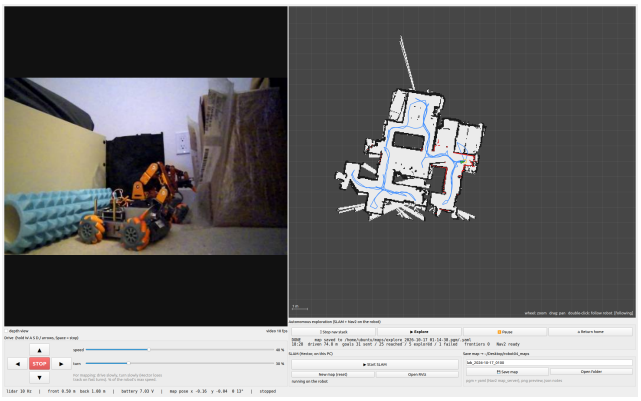


Figure 2: The operator application at the end of the run: camera and teleop (left); live map with the robot's SLAM path, Nav2 plan, goal, frontiers and start (right); exploration, SLAM and saving controls (bottom).

Termination and safety. When three searches in a row find no reachable frontier, the robot drives back to its start and the map is saved (map_server format). The run also ends at a time limit (45 min) and the robot returns home when the battery falls below 7.0 V. Every event (goal sent, reached, explored, failed, with timing, distance and pose) and a sample every 5 s (explored area, frontier count, battery) are written to a JSON-lines log; the results in section 8 are computed from it.

Operator control. The PC sends `start`, `pause`, `stop`, `home` and `save` on `/robot04/explore/control`. Any teleop key, the STOP button or the space bar sends `pause` first, which cancels the Nav2 goal and mutes the velocity adapter, so the operator always wins.

Table 2: CPU use on the Pi 5 while exploring (percent of one core, `top`, 2 s average).

Process	CPU %
aurora930 camera driver	26
explorer (Python, frontier search)	21
robot_node (Python, odometry + safety)	17
camera_bridge (JPEG)	9
bt_navigator	9
planner_server (NavFn + global costmap)	7
controller_server (RPP + local costmap)	6
ms200_node	5
behavior_server, velocity_smoother, lifecycle	11
hector_mapping	3
cmd_vel_adapter	3
Total (of 400)	≈ 150

6 Distributing the work

Two placements were considered: everything except the drivers on the PC (the robot as a sensor head), or everything that closes a loop on the robot (the PC as a viewer). The first mapping sessions ran Hector on the PC and worked: a scan is about 2 KB, 20 KB/s over Wi-Fi, and the measured extra latency was about 6 ms. For autonomous driving the second placement was chosen because a Wi-Fi drop of more than 0.5 s would otherwise stop the robot (robot_node's command timeout) or, worse, leave the controller acting on stale localisation; with everything on board a drop costs only the view. The cost is CPU on the Pi, measured in table 2: the whole stack uses about 1.5 of the Pi's 4 cores, at 52 °C without throttling. The camera driver and the explorer's Python search are the largest single loads; the explorer (and with it the global planner) is the natural candidate to move to the PC if the Pi ever becomes the bottleneck, because neither is time-critical. It was not moved because Humble's and Jazzy's `NavigateToPose` action types differ (Jazzy adds error fields to the result), so a Humble client and a Jazzy server do not interoperate reliably.

Middleware. The PC used Cyclone DDS and the robot's services Fast DDS. That worked for the sensor topics, but two problems disappeared only when the robot's services were switched to Cyclone as well: the camera bridge stopped receiving frames after the camera driver had been restarted (a new process subscribing to the same topic received them; Fast DDS shared-memory transport is the likely cause, and UDP-only transport also fixed it), and the explorer's state messages arrived at the PC garbled. Deserialisation warnings that remain between Humble and Jazzy come from `/parameter_events`, whose type differs between the two distributions, and are harmless.

7 Calibration: what the wheels did not say

The first autonomous runs produced maps with walls drawn twice at different angles. A rotation test showed why: commanded to turn left at 0.2 of full speed for 20 s, the robot's encoder odometry reported $+383^\circ$ and Hector reported -678°. Either the lidar was mirrored or the robot turned the other way from what its odometry believed.

Table 3: Rotation measured by scan alignment vs. encoder odometry, 1.5 s turns.

Command	Scan	Encoders	Note
before the fix ($+1/-1$, $b = 0.17$ m)			
turn $+0.25$	-51.2°	$+28.5^\circ$	wrong way
turn -0.25	$+44.8^\circ$	-28.8°	wrong way
after the fix ($-1/+1$, $b = 0.10$ m)			
turn $+0.25$	$+43.2^\circ$	$+46.8^\circ$	
turn -0.25	-47.2°	-50.3°	
turn $+0.40$	$+81.6^\circ$	$+85.3^\circ$	
turn -0.40	-86.4°	-88.5°	

To decide without trusting either, the rotation was measured from the lidar alone. With scans A (before) and B (after a short turn) as range vectors over the 450 rays, the rotation is the circular shift k^* that best aligns them,

$$k^* = \arg\min_k \operatorname{median}_i \left|A_i - B_{i+k}\right|, \qquad \Delta\psi = k^* \cdot 0.8^\circ, \quad (2)$$

over rays valid in both. Because the lidar's angle convention had been checked against the camera (objects to the left of the image appear at positive angles), this measurement is trustworthy. It said the robot turned *right* when told to turn left, which a before/after camera frame confirmed (the scene moved left in the image). A second test, the change of the range straight ahead after driving "forward", showed that the robot also drove backwards. Both motor signs had been inverted when the robot was first set up; because the odometry uses the same signs, it reported the commanded motion, not the real one, and nothing looked wrong until a sensor that does not share the motors' sign convention was compared.

Table 3 shows the measurements. After flipping both signs, the scan-measured rotation averaged 0.954 of the encoder estimate with track width 0.10 m, so the effective track width on carpet is 0.105 m, well below the 0.17 m between the track centres: a tracked robot turning in place skids, and the effective width absorbs that. The linear speed was measured with the encoders at 0.3 and 0.6 of full speed (0.047 and 0.091 m/s, i.e. $0.153\,f$ m/s) and checked against the lidar range to a wall; below about 0.1 of full speed the tracks do not move.

8 Results

8.1 Simulation

The explorer's frontier logic was first run offline (`tools/sim_explore.py`): a synthetic 10×7 m apartment with five rooms, a ray-cast 360° lidar with 450 rays and 8 m range, an occupancy grid written from the rays, and a robot that follows the geodesic path to the chosen goal and re-plans every 0.5 m. There is no SLAM error and no controller, so this tests the choice of goals and termination only. The robot mapped 100 % of the 64 m^2 of free space after 21.9 m of driving and 40 frontier goals; a frontier search took 36 ms on the laptop (fig. 3). Coverage rises quickly while the robot crosses open rooms and then flattens as it visits the last corners, the usual shape for greedy exploration.

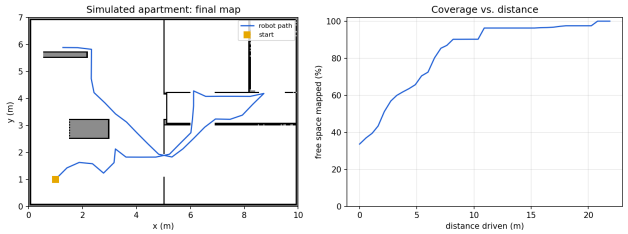


Figure 3: Offline simulation: final map and path (left), free space mapped against distance driven (right).

Table 4: The autonomous run, from the explorer's log.

Duration (start to end)	12.6 min
Mapped area (known cells)	47.3 m^2
Free / occupied area in the saved map	37.5 / 9.8 m^2
Map extent	10.9 × 13.2 m
Path length (SLAM poses, 5 cm steps)	74.0 m
Mean speed over the run	0.098 m/s
Frontier goals sent	30
reached	24
seen on the way (cancelled)	5
failed (blacklisted)	1
Operator interventions	0
Frontier search on the Pi, mean / max	50 / 58 ms
Candidate frontiers per search, mean	11.1
Mapping rate	3.8 m^2/min
Battery, start / end	7.40 / 7.08 V
How it ended	back at the start

8.2 Autonomous run in a home

The robot was placed in a carpeted hallway of a furnished apartment at night with the room lights on, the exploration service was started from the PC and Explore was pressed once; nobody touched the robot or the keyboard afterwards. Doors stood as they were left; there are no stairs on this floor. Table 4 summarises the run, fig. 4 shows the saved map with the path the robot chose and fig. 5 the mapped area over time. Of the 30 goals sent, 24 were reached and 5 were cancelled because the frontier behind them was seen on the way; 1 failed (timeout after 156 s). Mapped area grows fastest in the first two minutes, when every goal opens a new room to the 360° lidar, and creeps up afterwards as the robot visits the corners and gaps behind furniture one at a time; the number of open frontiers peaks at about four and a half minutes and falls to zero after 11.6 minutes, when the robot returned to its start. The single failure was a goal on the far side of a wall that the planner could reach only through unknown space; the controller made progress but not enough, and the goal was dropped after its timeout of 156 s. The mean speed over the run, 0.098 m/s, is close to the controller's cruise speed of 0.10 m/s, so the time was spent driving rather than waiting or recovering. The long white streak at the top of the map is free space seen along a few rays through a narrow gap (most likely a door left ajar); the frontier at its end is a ray or two wide, below the 0.30 m minimum frontier length, so the explorer ignored it, which is also why the room behind it is missing from the map.

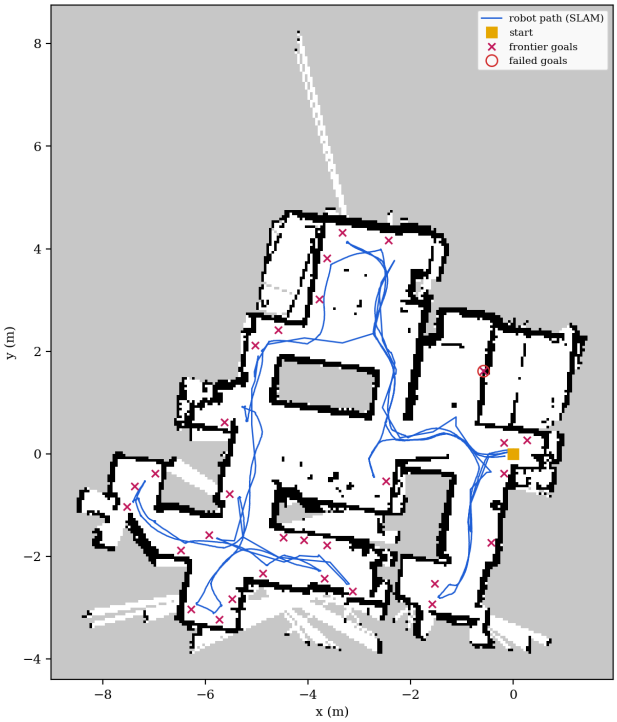


Figure 4: Map saved at the end of the autonomous run (5 cm cells; white free, black occupied, grey unknown), with the robot's path from SLAM, its start and the frontier goals it chose.

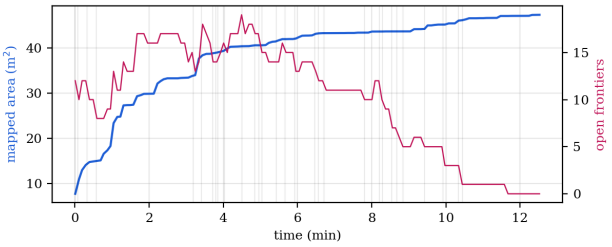


Figure 5: Mapped area (blue) and open frontiers (red) during the run; grey lines mark goals sent.

9 Lessons from bringing it up

The algorithms worked as published; the time went into the interfaces between them.

- **Check motion against a sensor that does not share its conventions.** Wheel odometry computed with the same signs as the motor commands will always agree with the commands. Two minutes of scan alignment found what weeks of teleop had not.
- **A controller cannot run faster than its costmap.** With the local costmap at 5 Hz and the controller at 10 Hz, Nav2's controller waited for the costmap and ran at 5–6 Hz, which looked like an overloaded CPU. The Pi was at 40 %.
- **SLAM time stamps lag.** Hector's `map→odom` carries the scan time; with Nav2's default transform tolerance the controller intermittently could not transform the robot pose and aborted goals.
- **Namespaces.** An action client created in namespace `/robot04` with the relative name `navigate_to_pose` looks for `/robot04/navigate_to_pose`; the explorer

waited for a server that was there all along under the root namespace.
- **Frontiers that cannot be cleared.** Reaching a frontier does not always remove it; without blacklisting reached goals the explorer looped.
- **Mixed DDS implementations** are supported in principle and fragile in practice; using the same one on every machine removed two unrelated faults at once.

10 Reproducing the run

All code is in the repository given below. On the robot, after the drivers' boot service is running:

```
sudo apt install ros-jazzy-navigation2 ros-jazzy-nav2-bringup \
    ros-jazzy-laser-geometry python3-scipy
cd ~/ros2_ws/src && git clone \
  https://github.com/RobotX-Workshops/hector_slam_ros2.git
# copy ros2/mentorpi_nav from the repository here, then
cd ~/ros2_ws && colcon build --packages-up-to \
  hector_mapping mentorpi_nav --cmake-args -DCMAKE_BUILD_TYPE=
      Release
cp src/mentorpi_nav/systemd/robot-explore.service \
   ~/.config/systemd/user/
systemctl --user daemon-reload; sudo loginctl enable-linger
     ubuntu
```

On the PC, `pc/robot04-mapper/run.sh` opens the application of fig. 2; *Start nav stack on robot* and *Explore* start a run. Afterwards the log (`~/explore_logs`) and the saved map (`~/maps`) are turned into the figures and tables of section 8 by `tools/analyze_run.py` and `make_results_tex.py`; `tools/sim_explore.py` reproduces fig. 3.

11 Limitations and future work

The 2D lidar sees a horizontal plane 12 cm above the floor: it cannot see a drop-off (stairs), low obstacles such as cables or flat toys, or overhangs such as table tops above the robot's height but below its camera. In this house there were no stairs on the mapped floor; elsewhere a no-go zone or a depth-camera cliff check is needed before the robot is left alone. Hector has no loop closure, so a long loop through several rooms can leave a visible seam; SLAM Toolbox would add it at a higher CPU cost. The explorer is greedy and does not trade information gain against travel; a next step is to weigh frontiers by the unknown area expected behind them and to use the depth camera for 3D obstacles.

12 Conclusion

A low-cost tracked robot with a Raspberry Pi 5 and an entry-level 360° lidar can map a home on its own with standard ROS 2 components, provided the parts are configured for each other: a lidar driver, Hector SLAM, Nav2 with Regulated Pure Pursuit and layered costmaps, a velocity adapter for a motor controller that speaks in fractions, and a frontier explorer that blacklists what it cannot clear. Running every loop on the robot and using the PC only to watch and steer means a Wi-Fi drop costs the operator's view rather than the robot's control loop (by design; dropouts were not provoked in this study), at a cost of about 1.5 of the Pi's four cores. The most consequential fault was not in any algorithm but in the robot's motor signs, and it was found by the simplest test: compare what the wheels say with what the lidar sees.

Code. `https://github.com/hmarthens1/rpi5-robot-board`: `ros2/mentorpi_robot` (drivers), `ros2/mentorpi_nav` (explorer, adapter, Nav2 configuration, simulation), `pc/robot04-mapper` (operator application).

References

[1] S. Kohlbrecher, O. von Stryk, J. Meyer, U. Klingauf, "A flexible and scalable SLAM system with full 3D motion estimation," *IEEE Int. Symp. Safety, Security and Rescue Robotics (SSRR)*, 2011.

[2] G. Grisetti, C. Stachniss, W. Burgard, "Improved techniques for grid mapping with Rao-Blackwellized particle filters," *IEEE Trans. Robotics*, 23(1), 2007.

[3] W. Hess, D. Kohler, H. Rapp, D. Andor, "Real-time loop closure in 2D LIDAR SLAM," *IEEE ICRA*, 2016.

[4] S. Macenski, I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, 6(61), 2021.

[5] S. Macenski, F. Martín, R. White, J. Ginés Clavero, "The Marathon 2: A navigation system," *IEEE/RSJ IROS*, 2020.

[6] D. V. Lu, D. Hershberger, W. D. Smart, "Layered costmaps for context-sensitive navigation," *IEEE/RSJ IROS*, 2014.

[7] S. Macenski, S. Singh, F. Martín, J. Ginés, "Regulated pure pursuit for robot path tracking," *Autonomous Robots*, 47, 2023.

[8] B. Yamauchi, "A frontier-based approach for autonomous exploration," *IEEE CIRA*, 1997.

[9] H. H. González-Baños, J.-C. Latombe, "Navigation strategies for exploring indoor environments," *Int. J. Robotics Research*, 21(10–11), 2002.

[10] M. Juliá, A. Gil, O. Reinoso, "A comparison of path planning strategies for autonomous exploration and mapping of unknown environments," *Autonomous Robots*, 33, 2012.

[11] H. Umari, S. Mukhopadhyay, "Autonomous robotic exploration based on multiple rapidly-exploring randomized trees," *IEEE/RSJ IROS*, 2017.

[12] J. Hörner, "Map-merging for multi-robot system," Bachelor's thesis, Charles University, Prague, 2016 (`explore_lite`).

[13] M. Hakizimana, "Lab 03: encoder motors and speed control," `rpi5-ros2-rrclite` lab notes, 2026.